

COMPILER DESIGN PROJECT

Code Summarization Using AST and NLP

Weekly Progress Report: Week 7

User Interface & Command-Line Tool Development

Submitted to: Ms. Preeti Soni

Course: Compiler Design

Date: 3 March 2026

WEEK 7

Command-Line Interface Development

Overview

This week focused on making the AST-based code summarization tool accessible to end users through a command-line interface (CLI). The objective was to wrap the core analysis engine (developed in Weeks 5-6) with an interactive shell that provides users with full control over which files to analyze, where to save reports, and how to handle batch operations. The result is a production-ready tool that can be integrated into security audit workflows and CI/CD pipelines.

1 Command-Line Interface Architecture

The CLI tool (`main.py`) serves as the entry point for all user interactions. It provides two primary modes of operation: single-file analysis and batch directory analysis. The design follows the principle of progressive disclosure — users are prompted for decisions only when necessary, and defaults are provided for common workflows.

1.1 Core Components

Component	Function	Responsibility
<code>analyze_file()</code>	Single file analysis	Validates file, reads source code, generates summary, handles save option
<code>analyze_directory()</code>	Batch analysis	Discovers all .py files, prompts for save location, analyzes each file
<code>main()</code>	Entry point	Presents menu, routes user choice to appropriate function
User prompts	Interactive control	Collects user decisions (file path, save location, continue/abort)

2 User Interaction Flow

The CLI is designed to be intuitive for users unfamiliar with the codebase. It presents a simple menu-driven interface with clear options and helpful prompts at each decision point.

2.1 Main Menu Flow

Welcome to the Python Code Summarizer!

What do you want to do?

- (1) Summarize a file
- (2) Summarize a directory
- (3) Exit

Enter your choice:

2.2 Single File Analysis Flow

Step	Action	User Control
1	Path Input	User provides file path
2	Validation	Tool checks if file exists and is .py
3	Analysis	AST extraction and summarization
4	Display	Summary printed to console
5	Save Option	Prompt: Save to file? (y/n)
6	Save Location	Default: summaries/ folder

3 Input Validation and Error Handling

Robust input validation ensures the tool handles edge cases gracefully and provides clear error messages.

3.1 File Validation Checks

Validation Type	Check Performed	Error Message
File Exists	<code>os.path.isfile(file_path)</code>	Error: Path is not valid
File Extension	<code>file_path.endswith('.py')</code>	Warning: Not a Python file
File Not Empty	<code>sourcecode.strip() != ""</code>	File is empty
Syntax Valid	<code>ast.parse()</code>	Handled by summarizer
Encoding Valid	UTF-8 decoding	UnicodeDecodeError if invalid

4 File I/O Operations and Save Management

4.1 Filename Generation Logic

```
# Original file: examples/sample_web_app.py
# Generated summary: summary_sample_web_app.txt

base_name = os.path.basename(file_path)
output_filename = f"summary_{base_name.replace('.py', '.txt')}"
```

5 Batch Processing and Directory Traversal

5.1 Directory Traversal Implementation

```
for root, dirs, files in os.walk(dir_path):
    for file in files:
        if file.endswith('.py'):
            python_files.append(os.path.join(root, file))
```

5.2 Batch Analysis Statistics

```
BATCH ANALYSIS COMPLETE
Successfully analyzed: 12 files
Failed: 2 files
```

6 Key Features Implemented

- Interactive menu system
- Real-time feedback messages
- Flexible save options
- Graceful error handling

7 Example Usage Scenarios

7.1 Scenario 1: Single File

```
$ python main.py
Enter your choice: 1
Enter file path: sample.py
Summarizing file...
Save report? (y/n): y
Report saved.
```

7.2 Scenario 2: Batch Mode

```
$ python main.py
Enter your choice: 2
Enter directory path: src/
Found 3 Python files
Batch Analysis Complete
```

8 Code Quality and Design Principles

8.1 Separation of Concerns

- `analyze_file()` handles single-file logic
- `analyze_directory()` handles batch orchestration
- `main()` handles user interaction

8.2 Extensibility

Future improvements may include:

- JSON/HTML output formats
- Filtering options
- Progress bars
- Command-line argument parsing

9 Week 7 Deliverables Summary

Deliverable	Status	Description
Command-Line Interface	Complete	main.py with menu interaction
Single File Analysis	Complete	Validation and save option
Batch Directory Analysis	Complete	Recursive discovery
Error Handling	Complete	Graceful failure management
Save Management	Complete	Auto filename generation

10 Conclusion

Week 7 successfully transformed the AST-based code summarization tool from a developer-facing library into a user-friendly command-line application. The interactive CLI provides full control over file selection, batch processing, and output management while maintaining robust error handling and clear user feedback. The tool is now ready for real-world deployment and serves as a strong foundation for further enhancements.